ДИСЦИПЛИНА	Технологии разработки серверных приложений
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Текущий контроль
ПРЕПОДАВАТЕЛЬ	Дворецкий Артур Геннадьевич
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Ссылка на материал:

https://github.com/dv0retsky/fastapi-tutorial/blob/main/FAPI_KR3/FAPI_KR3.md

---

Контрольная работа №3 🐝

🦄 Как сдавать работу

1. Выполните задания в **отдельном репозитории** (рекомендуется: один репозиторий на одну контрольную работу).
2. Загрузите исходный код в репозиторий (`commit` + `push`).
3. В **СДО** прикрепите ответ в виде **ссылки на репозиторий с решением**.

> **Рекомендации к оформлению репозитория:**
>
> - Проект должен **запускаться** и быть готов к проверке (например: `main.py` / `app.py` , `models.py` , `database.py` и др. — по необходимости).
> - Добавьте зависимости: `requirements.txt` или `pyproject.toml` .
> - В `README.md` кратко опишите:
> - как установить зависимости и запустить приложение (`uvicorn ... --reload`);
> - как тестировать ключевые эндпоинты (можно командами `curl`).
> - Если используются переменные окружения (например, `MODE` , `DOCS_USER` , `DOCS_PASSWORD` и т.п.) — добавьте файл **`.env.example`** и **не публикуйте реальные секреты**. Файл `.env` должен быть в `.gitignore` .

🌟 Задание 6.1

Реализуйте защищенную базовой аутентификацией конечную точку **FastAPI** `/login`, которая принимает запросы `GET`.

1. Конечная точка должна аутентифицировать пользователя на основе предоставленных учетных данных.
2. Используйте зависимость, чтобы проверить правильность имени пользователя и пароля.
3. Если учетные данные неверны, верните сообщение `HTTPExceptionc` кодом состояния `401` (то же самое возвращается, если учетные данные не предоставлены).
4. Если данные верны, верните секретное сообщение `"You got my secret, welcome"`
5. Попробуйте сначала авторизоваться с неправильными данными, а потом введите корректные данные. Для получения такой возможности (повторно авторизоваться) изучите информацию про необходимость добавления заголовка `WWW-Authenticate`, чтобы браузер снова отображал приглашение для входа в систему.

🌟 Задание 6.2

Расширьте приложение **FastAPI**, реализовав **безопасную аутентификацию** с использованием хеширования паролей и защиты от тайминг-атак. Вам потребуется переиспользовать код из предыдущего задания, следуя принципу **Don't Repeat Yourself (DRY)**.

1. Создание моделей данных

- Опишите Pydantic-модель **UserBase** с полями:
  - `username:` строка, обязательное поле
- Опишите Pydantic-модель **User**, которая наследуется от **UserBase**, с полями:
  - `username:` строка, обязательное поле (будет отнаследовано)
  - `password:` строка, обязательное поле
- Опишите Pydantic-модель **UserInDB**, которая наследуется от **UserBase**, с полями:
  - `username:` строка, обязательное поле (будет отнаследовано)
  - `hashed_password:` строка, обязательное поле

> **Примечание:** в модели для хранения в базе (`UserInDB`) не должно быть поля с открытым паролем — только его хеш.

2. Настройка PassLib

Инициализируйте `CryptContext` с алгоритмом `Bcrypt` для генерации и проверки хешей паролей.

3. Зависимость аутентификации

Реализуйте функцию `auth_user`, которая:

- Извлекает `username` и `password` из заголовка `Authorization` через `HTTPBasicCredentials`.
- Находит пользователя в in-memory базе (словарь/массив `fake_users_db`).
- Проверяет корректность пароля с помощью метода `verify` из вашего `CryptContext`.
- Для сравнения логин-строк используйте `secrets.compare_digest()`.
- При успехе возвращает объект юзера. В случае неудачи выбрасывает `HTTPException` со статусом `401` и заголовком `WWW-Authenticate: Basic`.

4. Реализация маршрутов `/register` и `/login`

Добавьте POST-эндпоинт `/register`, который:

- Принимает тело запроса в формате JSON, соответствующее модели **User**.
- Генерирует хеш пароля с помощью `CryptContext`.
- Создаёт объект по модели **UserInDB** и сохраняет его в in-memory базе `fake_users_db`.
- При успешной регистрации возвращает сообщение об успешном добавлении пользователя

Добавьте GET-эндпоинт `/login`, который принимает автоматически подставленный через `Depends` результат работы `auth_user` и возвращает JSON с приветствием:

- При успешной аутентификации: статус **200** и тело ответа вида:

```
{ "message": "Welcome, <username>!" }
```

- При ошибке: статус **401** и заголовок `WWW-Authenticate: Basic`.

5. Тестирование через `curl`

```
# Регистрация
curl -X POST -H "Content-Type: application/json" -d
'{"username":"user1","password":"correctpass"}'
http://localhost:8000/register

# Успешный логин
curl -u user1:correctpass http://localhost:8000/login

# Неверный пароль
curl -u user1:wrongpass http://localhost:8000/login
```

6. Обработка ошибок (опционально)

- Если в заголовках отсутствуют `username` или `password`, FastAPI автоматически вернёт 401.
- Для кастомных сценариев используйте `HTTPException` с добавлением `headers={"WWW-Authenticate": "Basic"}`.

Дополнительные материалы

1. **PassLib: хеширование паролей** (оф. дока на англ.)
2. **PassLib: CryptContext** (оф. дока на англ.)
3. **Python: secrets.compare_digest** (оф. дока на англ.)
4. **RFC 7235: HTTP Authentication** (стандарт на англ.)
5. **Хабр про хэширование** (статья на русском)
6. **FastAPI: зависимости (Depends)** (оф. дока на русском)
7. **Pydantic: модели и наследование** (оф. дока на англ.)

🌟 Задание 6.3

Расширьте предыдущее приложение FastAPI, добавив управление документацией в зависимости от режима окружения (`DEV` / `PROD`). Мы условно готовим приложение к развёртыванию в продакшене: в проде у нас вся документация API должна быть отключена, а в деве мы защищаем ссылку `/docs` базовой аутентификацией (чтобы туда доступ был только по логину/паролю).

1. Конфигурация окружения
Режим работы приложения определяется переменной окружения `MODE` (допустимые значения: `DEV`, `PROD`). Данные для аутентификации в документации (только для `DEV`) хранятся в переменных окружения: `DOCS_USER` и `DOCS_PASSWORD`.

2. Защита документации в DEV-режиме
В режиме `DEV`:

- Эндпоинты `/docs` и `/openapi.json` должны требовать базовую аутентификацию. Эндпоинт `/redoc` должен быть скрыт.
- Для проверки учетных данных используйте `secrets.compare_digest()` для защиты от тайминг-атак.
- При неудачной аутентификации возвращайте статус `401` с заголовком `WWW-Authenticate: Basic`.
- Сами эндпоинты должны быть исключены из схемы OpenAPI (`include_in_schema=False`).

3. Отключение документации в PROD-режиме
В режиме `PROD`:

- Полностью скройте эндпоинты `/docs`, `/openapi.json`, `/redoc`.
- При обращении к ним должен возвращаться статус `404 Not Found`.

4. Реализация

- Используйте условные проверки режима при инициализации приложения.
- Для переопределения стандартных эндпоинтов документации добавьте кастомные маршруты с зависимостями (dependencies).
- Убедитесь, что в PROD-режиме схема OpenAPI не генерируется или недоступна.

5. Тестирование

DEV-режим:

```
curl -u valid_user:valid_password http://localhost:8000/docs
```

PROD-режим:

```
curl http://localhost:8000/docs  # Должен вернуть 404
```

Примечание:

- Для работы с переменными окружениями используйте `python-dotenv`, `environs` или `pydantic-settings`.
- Стандартные маршруты документации FastAPI должны быть переопределены.
- Не забудьте обработать кейс, когда переменная `MODE` имеет недопустимое значение.

🌟 Задание 6.4

Для этой задачи программирования вам необходимо реализовать аутентификацию на основе JWT в приложении FastAPI. Используйте библиотеку `PyJWT` для генерации и проверки токенов.

Требования:

1. Создайте **конечную точку** FastAPI `/login`, которая принимает запросы POST с полезной нагрузкой JSON, содержащей поля "имя пользователя" (`username`) и "пароль" (`password`). Конечная точка должна аутентифицировать пользователя на основе предоставленных учетных данных.

2. Если учетные данные действительны, **сгенерируйте токен JWT** с соответствующим сроком действия и верните его в ответе.
3. Если учетные данные неверны, верните соответствующий **ответ об ошибке**.
4. Создайте защищенную конечную точку FastAPI `/protected_resource`, **для которой требуется аутентификация** с использованием **JWT**. Пользователи должны включать сгенерированный токен в заголовок `Authorization` своих запросов для доступа к этой конечной точке.
5. **Проверьте токен JWT** в заголовке `Authorization` для каждого запроса к `/protected_resource`. Если токен действителен, разрешите доступ к конечной точке и верните ответ, указывающий на успешный доступ.
6. Если токен недействителен, срок действия истек или отсутствует, верните соответствующий **ответ об ошибке**.

Примечание: Вы можете предположить существование гипотетической функции `authenticate_user(username: str, password: str) -> bool`, которая проверяет предоставленные "имя пользователя" и "пароль" по базе данных пользователя и возвращает `True`, если учетные данные действительны, и `False` в противном случае (или создать заглушку такой функции, которая при помощи модуля `random.choice` возвращает True или False).*

Пример запроса:

```
POST /login
Content-Type: application/json

{
  "username": "john_doe",
  "password": "securepassword123"
}
```

Пример ответа (200 OK):

```
{
  "access_token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6
IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_
adQssw5c"
}
```

Пример ответа (401 Unauthorized):

```
{
  "detail": "Invalid credentials"
}
```

> ⚠️ **Warning**
>
> ***Примечание:*** Поскольку это упрощенная задача программирования, **крайне важно принять дополнительные меры безопасности** и следовать рекомендациям для реальных приложений. Кроме того, обработка токенов, механизмы обновления и отзыв токенов являются важными аспектами, которые необходимо учитывать при создании готовой к работе системы аутентификации!

🌟 Задание 6.5

Расширьте и улучшите предыдущее решение с JWT-аутентификацией в FastAPI, реализовав:

1. Конечную точку регистрации

- **Маршрут:** `POST /register`
- **Вход:** JSON с полями `username` (строка) и `password` (строка).
- **Логика:**
  - Если пользователь с таким `username` уже существует в фейковой БД, верните ошибку **409 Conflict** с сообщением `{"detail": "User already exists"}`.
  - Если пользователя нет, сохраните его в БД с **хэшем пароля** (используйте `passlib`). Верните статус **201 Created** с сообщением `{"message": "New user created"}`.

2. Обновлённую логику аутентификации

- **Маршрут:** `POST /login`
- **Логика:**
  - Если пользователя нет в БД, верните **404 Not Found** с сообщением `{"detail": "User not found"}`.
  - Если пользователь есть, сравните хэш принятого пароля с сохранённым хэшем. При поиске в БД для сравнения `username` используйте **модуль** `secrets`, чтобы избежать тайм-атак.
  - При успешной проверке верните JWT-токен (`access_token`) с полезной нагрузкой `{"sub": username}`.
  - Если пароль неверен, верните **401 Unauthorized** с сообщением `{"detail": "Authorization failed"}`.

3. Внедрите Rate Limiter (ограничение количества запросов)

- Установите ограничения:
  - **/register:** 1 запрос в минуту (защита от спама).

    - **/login:** 5 запросов в минуту (ограничение попыток ввода пароля).
- При превышении лимита возвращайте **429 Too Many Requests** с сообщением `{"detail": "Too many requests"}`, если будете самостоятельно реализовывать такую логику (при использовании библиотек уже встроена логика ответа с этим статус кодом).

Требования к реализации

- Используйте `passlib` для хэширования паролей (рекомендуется алгоритм `bcrypt`).
- Для сравнения username применяйте `secrets.compare_digest()`.
- Для `rate-limiting` можете использовать библиотеку `slowapi` или `fastapi-limiter`.
- Храните пользователей в памяти (например, в словаре или списке) с хэшами паролей.

Примеры запросов и ответов

Регистрация:

```
POST /register
Content-Type: application/json

{ "username": "alice", "password": "qwerty123" }

Ответ (201):
{ "message": "New user created" }

Ответ (409):
{ "detail": "User already exists" }
```

Логин:

```
POST /login
Content-Type: application/json

{ "username": "alice", "password": "qwerty123" }

Ответ (200):
{ "access_token": "eyJhbGci...", "token_type": "bearer" }

Ответ (404):
{ "detail": "User not found" }

Ответ (401):
{ "detail": "Authorization failed" }
```

Protected Resource:

```
GET /protected_resource
Authorization: Bearer eyJhbGci...

Ответ (200):
{ "message": "Access granted" }
```

Дополнительные материалы для изучения

- [Passlib: официальная документация](#) — полный справочник по библиотеке.
- [secrets.compare_digest в официальной документации Python](#) — описание функции и примеры использования.
- [Документация SlowAPI](#) — документация slowapi.
- [fastapi-limiter на GitHub](#) — официальный репозиторий с инструкциями и примерами (требует Redis'a).
- [Сравнение алгоритмов ограничения частоты запросов](#) — анализ популярных алгоритмов (fixed window, sliding window, token bucket).
- [JWT-аутентификация в FastAPI: полное руководство](#) — практический пример реализации access/refresh-токенов.
- [OAuth2 и JWT в FastAPI: официальный туториал](#) — встроенные механизмы безопасности FastAPI.
- [Securing FastAPI with JWT Token-based Authentication](#) — англоязычное руководство с примерами.

> **Примечание:** В реальных проектах обязательно используйте `HTTPS`, храните секретные ключи в защищённых хранилищах (например, `Vault`) и реализуйте механизм обновления токенов. Удачи в разработке! 🚀

🌟 Задание 7.1

Для этой задачи программирования вам необходимо реализовать управление доступом на основе ролей в приложении FastAPI. Предположим, что аутентификация на основе JWT уже реализована, и пользователям назначаются роли в процессе регистрации.

Требования:

1. Создайте как минимум три роли: **"администратор"**, **"пользователь"** и **"гость"** (`admin`, `user`, `guest`). Определите соответствующие разрешения для каждой роли. Например, "администратор" может иметь полные разрешения CRUD, в то время как "пользователь" может читать и обновлять ресурсы, а "гость" имеет ограниченный доступ только для чтения.

2. Реализуйте **авторизацию** на основе ролей для ваших конечных точек FastAPI. Используйте внедрение зависимостей или авторизацию на основе декоратора, чтобы проверить роль пользователя, прежде чем разрешить доступ к определенным конечным точкам.
3. Создайте конечную точку FastAPI `/protected_resource`, для которой требуется аутентификация. Убедитесь, что только пользователи с соответствующей ролью (например, "администратор" или "пользователь") могут получить доступ к этой конечной точке.
4. Для каждой роли создайте определенные конечные точки API, которые демонстрируют разрешения этой роли. Например, роль "администратор" может иметь конечную точку для создания ресурса, в то время как роль "пользователь" может только читать и обновлять существующие ресурсы.
5. Протестируйте свою реализацию RBAC, отправляя запросы к различным конечным точкам с разными ролями пользователей. Убедитесь, что доступ надлежащим образом ограничен в зависимости от роли пользователя.

> **Примечание:** *Для этой задачи вы можете использовать **словари Python** или простое хранилище данных в памяти для хранения ролей и связанных с ними разрешений. В реальном сценарии вы обычно интегрируете RBAC с базой данных вашего приложения.*

🌟 Задание 8.1

1. Создайте таблицу users в SQLite с полями:

- `id` (INTEGER, первичный ключ с автоинкрементом)
- `username` (TEXT, обязательное поле)
- `password` (TEXT, обязательное поле)

2. Реализуйте эндпоинт `POST /register`, который будет принимать `username` и `password` и сохранять их в таблицу users.

Пошаговая инструкция:

1. В файле `database.py` настройте подключение к базе данных.
2. Напишите скрипт для создания таблицы `users`. Запустите его **один раз**, чтобы таблица создалась.
3. В `main.py` создайте Pydantic-модель `User` с полями `username` и `password`.
4. Добавьте эндпоинт `POST /register`, который:
   - Подключается к базе через `get_db_connection()`
   - Вставляет новые данные в таблицу `users`
   - Фиксирует изменения (`commit()`)
   - Закрывает соединение
   - Возвращает `{"message": "User registered successfully!"}`

5. Протестируйте API с помощью команды:

```
curl -X 'POST' 'http://127.0.0.1:8000/register' \
-H 'Content-Type: application/json' \
-d '{"username": "test_user", "password": "12345"}'
```

Требования к коду:

- Используйте синтаксис из лекции (без SQLAlchemy)
- Названия таблицы и полей должны точно совпадать с условием (`users` , `username` , `password`)
- Не забудьте про `IF NOT EXISTS` при создании таблицы
- Пароль пока можно хранить в открытом виде (без шифрования)

Проверьте, что:

- Таблица создается без ошибок
- Эндпоинт возвращает успешный ответ
- Данные появляются в базе (можно проверить через DB Browser for SQLite)

🌟 Задание 8.2

Для этой задачи программирования вам необходимо интегрировать FastAPI с базой данных и выполнить базовые операции CRUD с определенным ресурсом.

Требования:

1. **Выберите любую поддерживаемую базу данных** (например, SQLite, PostgreSQL, MySQL или MongoDB) и установите необходимый драйвер базы данных для FastAPI (рекомендуется выбирать ту, с которой больше всего вероятность работать в будущем, либо ту, с которой вы ещё не умеете работать).
2. **Создайте модель данных (схему)** для простого ресурса (например, элемента " `Todo` " (воображаемый список дел), который включает такие поля, как " `id` ", " `title` " (заголовок), " `description` " (описание) и " `completed` " (завершено)).
3. **Реализуйте конечную точку FastAPI** для создания нового элемента " `Todo` ". Конечная точка должна принимать POST-запросом полезную нагрузку JSON, содержащую поля "заголовок" и "описание". После успешного создания верните созданный элемент " `Todo` " в ответе (по умолчанию у нового элемента статус (признак) завершено равен `False`).
4. **Реализуйте конечную точку FastAPI** для извлечения одного элемента " `Todo` " на основе его " `id` ". Конечная точка на GET-запрос должна возвращать соответствующий элемент " `Todo` ", если он найден, или соответствующий ответ об ошибке, если элемент не существует.
5. **Реализуйте конечную точку FastAPI** для обновления существующего элемента " `Todo` " на основе его " `id` ". Конечная точка должна принимать полезную нагрузку

JSON (`PUT/POST-запрос`), содержащую поля "заголовок", "описание" и "завершено". Обновите соответствующий элемент " `Todo` " в базе данных и верните обновленный элемент в ответе.

6. **Реализуйте конечную точку FastAPI** для удаления элемента " `Todo` " на основе его " `id` ". Если элемент успешно удален, верните сообщение об успешном завершении в ответе.

Пример POST-запроса create (Создать Todo):

```
POST /todos
Content-Type: application/json

{
  "title": "Buy groceries",
  "description": "Milk, eggs, bread"
}
```

Пример ответа (201 Created):

```
{
  "id": 1,
  "title": "Buy groceries",
  "description": "Milk, eggs, bread",
  "completed": false
}
```

Пример GET-запроса read (Получить Todo - ID: 1):

```
GET /todos/1
```

Пример ответа (200 OK):

```
{
  "id": 1,
  "title": "Buy groceries",
  "description": "Milk, eggs, bread",
  "completed": false
}
```

Дополнительные материалы

- FastAPI с SQL-базами данных (например, SQLite, PostgreSQL):
  - **Официальное руководство FastAPI по работе с SQL-базами данных**
  - **Пример использования FastAPI с raw SQL и PostgreSQL**
- FastAPI с NoSQL-базами данных (например, MongoDB):

    - Руководство по созданию CRUD-приложения с FastAPI и MongoDB (Motor)
    - Руководство по созданию CRUD-приложения с FastAPI и MongoDB (Beanie)
    - Пример реализации CRUD-операций с FastAPI и MongoDB
- Справочники по синтаксису SQL:
    - SQL Syntax - W3Schools
    - SQL Syntax Overview - Tutorialspoint
    - SQL Syntax - SQL Tutorial
    - Быстрое знакомство с SQL (хабр)
    - Тренажер SQL

Made with ❤️ by **dv0retsky**